WEEK 7 – VAPT

Web Application Testing with Burp Suite

Vulnerability Assessment & Penetration Testing Report
Prepared by: Krushna Chandak
Program: DG Interns Hub Cybersecurity Internship
Target: Damn Vulnerable Web Application (DVWA), local host 127.0.0.1:8081
Testing environment: Linux Mint, Firefox, Burp Suite Community Edition
Assessment period: October 2026 | Scope limited to a deliberately vulnerable training application

Important scope statement: All active testing documented here was performed against the local DVWA lab. No production website or third-party system was tested. Results demonstrate learning in a controlled environment and should not be interpreted as an assessment of a real organization.

1. Executive Summary

This report records the Week 7 web-application testing workflow using Burp Suite Community Edition. The work covered interception of HTTP/HTTPS traffic, identification of application routes and parameters, manual testing in Repeater, a small Intruder exercise, and directory/endpoint discovery. Evidence is based on requests and responses captured from the local DVWA instance.

Three vulnerability classes were demonstrated in the intentionally vulnerable DVWA application: SQL injection, reflected cross-site scripting (XSS), and stored XSS. The SQL injection tests showed a database error for a quote-based input and a Boolean-style input that returned multiple records. The reflected-XSS test executed a JavaScript alert from a request parameter. The stored-XSS test executed an alert from a guestbook entry that was stored by the application and rendered when the page was viewed.

The results are educational proof-of-concept observations, not evidence that the same flaws exist in an unrelated site. DVWA was configured to low security for the demonstrations, so findings illustrate insecure behavior rather than the difficulty of exploiting a hardened production application.

Key outcomes

Area	Outcome
Proxy and HTTPS	Burp listener and browser proxy verified; HTTPS request intercepted with existing trusted PortSwigger CA.
Mapping	Site map populated with login, information pages and vulnerability modules; request parameters documented.
Confirmed demonstrations	SQL injection, reflected XSS, stored XSS in local DVWA.
Other tests	Intruder two-payload exercise and a 14-path endpoint check; results interpreted conservatively.

2. Scope and Rules of Engagement

The sole target was the local DVWA application available at `http://127.0.0.1:8081`. Testing was restricted to this deliberately vulnerable lab. The browser proxy was configured to use `127.0.0.1:8080`, with Burp Proxy listening on that address. The work did not include scanning external hosts, public IP addresses, or systems belonging to other people.

In-scope activities

- Intercepting browser requests and responses in Burp Proxy.
- Reviewing the Target site map and recording observed routes and parameters.
- Replaying controlled requests in Repeater to compare baseline and test inputs.
- Executing a small Intruder payload set against the local lab only.
- Checking a predefined list of 14 common paths and interpreting HTTP status codes.
- Capturing screenshots for a reproducible report.

Out-of-scope activities

No testing was performed against real user accounts, third-party infrastructure, or production services. The directory check was a small, manually defined list and is not a comprehensive content discovery scan. No claim is made that all routes or vulnerabilities were found.

Safety and evidence handling

Screenshots of HTTP messages can contain cookies, session identifiers, CSRF tokens, and submitted credentials. These values should be redacted before public posting. A screenshot is retained as evidence of a local test, not as a reason to publish reusable session material.

Assessment limitations

DVWA is intentionally insecure and was tested at low security for the demonstrated modules. Results are specific to this setup and configuration. A real engagement would require written authorization, agreed scope, safe test accounts, change windows, and a documented process for handling sensitive data.

3. Methodology and Tooling

The workflow followed a simple evidence-led cycle: establish the authorized target, intercept a normal browser action, identify the request method and parameters, replay a baseline, change one input at a time, compare the response, and save the result. This reduces ambiguity because each test can be related to a specific request change.

Tools used

Burp Proxy: captured browser HTTP/HTTPS requests and responses. **Target / Site map:** organized the routes observed during browsing. **Repeater:** allowed individual requests to be replayed and modified while comparing responses. **Intruder:** sent a deliberately small two-payload set to demonstrate repeatable parameter testing. **Python path checker:** requested 14 predefined paths from the local DVWA host and recorded returned status codes. Firefox was the browser used for the local lab.

Evidence standards

A confirmed vulnerability requires more than a route existing or a response length changing. The strongest evidence in this exercise is observable behavior: SQL error/record changes for SQL injection, JavaScript alert execution for reflected XSS, and a stored guestbook entry triggering JavaScript when rendered for stored XSS. Input changes to the command module are reported only as input influence because they do not, by themselves, prove arbitrary command execution.

Repeatability

For each finding, the report describes the affected feature, test input, observed result, potential impact, and remediation. Screenshots are included where available. Exact cookies and tokens are intentionally not reproduced in this report because they are not necessary to explain the behavior.

Workflow summary

Step	Purpose
1 — Intercept	Confirm the browser request reaches Burp and capture the HTTP message.
2 — Map	Record routes, methods, parameters and relevant forms.
3 — Test	Replay in Repeater and vary one input in the local target.
4 — Validate	Observe response behavior and, for XSS, confirm execution in the browser.
5 — Report	Record proof, impact, limitations and defensive fixes.

4. Burp Setup, Interception and Application Mapping

Burp Suite Community Edition was already installed before this task. The listener was verified on `127.0.0.1:8080` and Firefox was configured to route HTTP and HTTPS through it. The existing PortSwigger CA certificate was already present in Firefox and trusted for identifying websites, so the certificate did not need to be reinstalled. A request to an HTTPS example site was intercepted without a browser certificate warning; local DVWA HTTP traffic was also captured.

Login request analysis

The login form generated a `POST /login.php` request with form fields for username, password, the Login action, and a user token. The request also carried cookies. The response captured during the login flow returned `302 Found` and a redirect location. A redirect by itself is not conclusive proof of successful authentication; the browser session and subsequent page behavior must be considered as well.

Site map and observed endpoints

The site map included the main pages `/index.php`, `/login.php`, `/logout.php`, `/about.php`, `/instructions.php`, `/security.php`, and `/setup.php`. Observed vulnerability modules included brute force, CAPTCHA, CSRF, command execution, file inclusion, SQL injection, blind SQL injection, upload, weak session ID, and reflected/stored/DOM XSS paths. These are routes observed while browsing the local DVWA app, not proof that each route contains a separate exploitable flaw.

Request parameters recorded

Examples included the SQLi GET parameters `id` and `Submit`; reflected XSS GET parameter `name`; stored XSS POST fields `txtName`, `mtxMessage`, and `btnSign`; file inclusion parameter `page`; and command module parameter `ip`. Recording method and parameter names makes later test results reproducible.

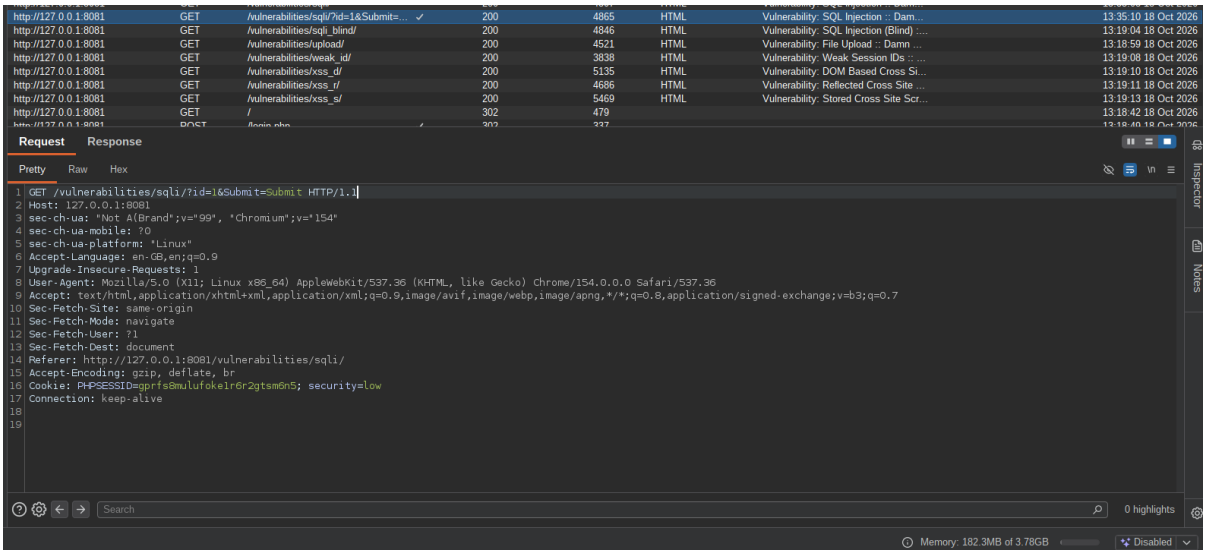


Figure 1. Example of a captured GET request and visible parameters in Burp.

5. Finding 1 — SQL Injection

Target feature: DVWA SQL Injection module, `/vulnerabilities/sqli/`
Evidence basis: Repeater response behavior in the local DVWA low-security configuration.
Assessment result: Confirmed in this deliberately vulnerable lab.

Description

SQL injection occurs when untrusted input is incorporated into a database query in a way that allows the input to alter the query's intended structure. In this test, the numeric-looking `id` parameter was changed from its baseline value. A quote-based test produced a database syntax error, indicating that the input reached SQL query construction without being safely handled. A Boolean-style input then returned multiple records, including the administrator record, instead of behaving like a single ordinary identifier lookup.

Reproduction summary

1. Open the SQL Injection module in local DVWA and capture the normal GET request in Burp.
2. Send the request to Repeater and note the baseline response for `id=1`.
3. Change only the `id` value to a single quote and send the request; observe the SQL syntax error in the response.
4. In the local training app only, use the Boolean-style test value already documented in the saved evidence and compare the returned records with the baseline.
5. Save the request and response screenshots, ensuring session tokens are redacted before publication.

Impact

If a comparable flaw existed in a real application, an attacker might influence query logic and access records beyond the intended lookup. Depending on database permissions and query structure, consequences could include unauthorized data exposure, authentication logic abuse, or data modification. The local demonstration does not establish the impact on any external application.

Remediation

Use parameterized queries/prepared statements so input is treated as data, not SQL syntax. Validate that identifiers are numeric and within expected ranges, apply least-privilege database accounts, avoid returning detailed SQL errors to users, and add regression tests for malformed and unexpected inputs. Input filtering alone is not a substitute for parameterized queries.

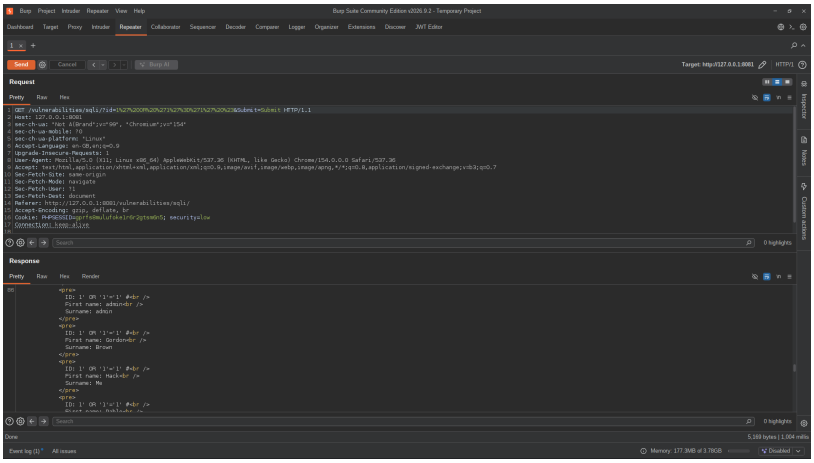


Figure 2. Boolean-style SQL injection response in the local DVWA test.

6. SQL Injection — Evidence and Interpretation

The evidence should be read as a comparison rather than as an isolated screenshot. The baseline request asks the application for a single record by ID. The quote-based test triggers a database error, and the Boolean-style test changes the result set. Together, these observations support the conclusion that the parameter is being used unsafely in a SQL query in the low-security training configuration.

Why the finding is considered confirmed

The conclusion is based on a change in application behavior that is consistent with SQL syntax being interpreted: an error after introducing a quote, followed by a result set that differs from the normal identifier lookup when a Boolean condition is supplied. A route name or a parameter existing would not be sufficient. The result is reproducible in DVWA, which intentionally includes this weakness.

Evidence quality and limitations

The test was not run against a production system. No claim is made that the database contents were fully enumerated, that data was modified, or that operating-system command execution was achieved. The report does not reproduce the full cookie header or session token. A stronger production assessment would record the exact application version, database type, security setting, sanitized request/response pairs, and a controlled retest after remediation.

Recommended validation after a fix

After implementing prepared statements, repeat the baseline lookup and the same malformed-input tests. The malformed input should be treated as an ordinary invalid value, not cause a database error, and should not change query structure or return records outside the intended identifier. Server-side logs should retain enough diagnostic information for developers without exposing SQL details in the browser response.

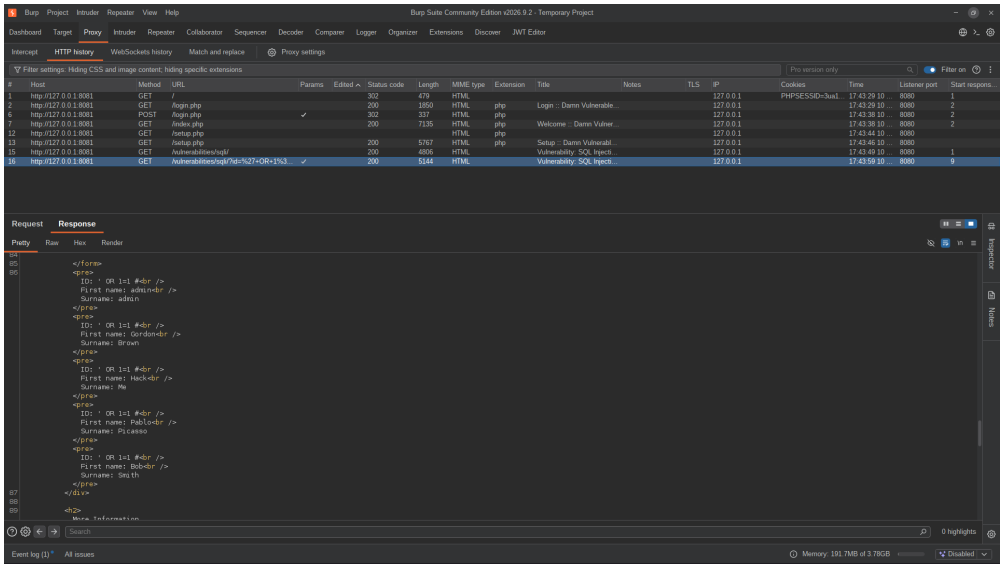


Figure 3. Supporting SQL injection response evidence captured during local testing.

7. Finding 2 — Reflected Cross-Site Scripting (XSS)

Target feature: DVWA Reflected XSS module, `/vulnerabilities/xss_r/`
Parameter: `name`
Assessment result: Confirmed in local DVWA.

Description

Reflected XSS occurs when data supplied in a request is returned in the response without appropriate context-aware output encoding, allowing a browser to interpret the data as executable markup or script. The test supplied a harmless JavaScript alert proof-of-concept through the reflected name parameter. Burp Repeater showed the supplied script returned in the response without safe encoding, and the browser then executed it and displayed an alert.

Reproduction summary

1. Browse to the local Reflected XSS module and intercept its GET request.
2. Send the request to Repeater and identify the `name` parameter.
3. Use the harmless alert payload documented in the local evidence and send the modified request.
4. Inspect the response for unsafe reflection of the supplied script.
5. Open the request in the lab browser and confirm the alert appears. Keep the test limited to the local DVWA target.

Impact

On a real vulnerable site, reflected XSS may allow attacker-controlled JavaScript to execute in the context of the site for a victim who follows a crafted link or submits a manipulated request. Depending on application behavior and browser protections, this could enable misleading page content, unauthorized actions available to the victim, or access to data exposed to scripts. This test only demonstrates script execution in DVWA.

Remediation

Apply context-aware output encoding wherever user-controlled values are inserted into HTML, attributes, JavaScript, or URLs. Prefer safe templating frameworks and DOM APIs that treat content as text. Validate input according to business requirements, use a restrictive Content Security Policy as defense in depth, and avoid inserting untrusted strings into dangerous HTML sinks.

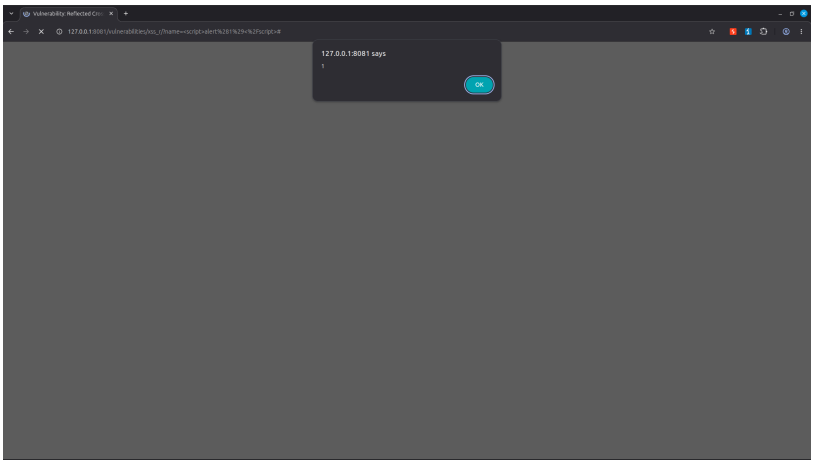


Figure 4. Browser alert confirming the reflected XSS proof-of-concept in local DVWA.

8. Reflected XSS — Request/Response Evidence

The browser alert is the clearest user-visible proof that the reflected input was interpreted as script. The Burp request and response provide the protocol-level context: the test value was supplied through a GET parameter and returned by the application. Both forms of evidence are useful because a response containing text that resembles a script is not always enough to prove execution; browser behavior closes that gap for this local demonstration.

Evidence handling

When sharing screenshots, ensure the target address remains visible enough to show that the test was performed on `127.0.0.1:8081`, while session identifiers and unrelated browser content are obscured if needed. The alert text itself should remain visible. Do not use a payload that collects credentials, sends data to an external host, or performs destructive actions; a local alert is sufficient to demonstrate the flaw.

Validation after remediation

After encoding is implemented, submit the same test string. It should be displayed as inert text or safely rejected, and the browser must not execute it. Test relevant output contexts separately because HTML body text, attributes, JavaScript strings, and URL contexts require different handling. A Content Security Policy may reduce risk but should not be the primary fix for unsafe output encoding.

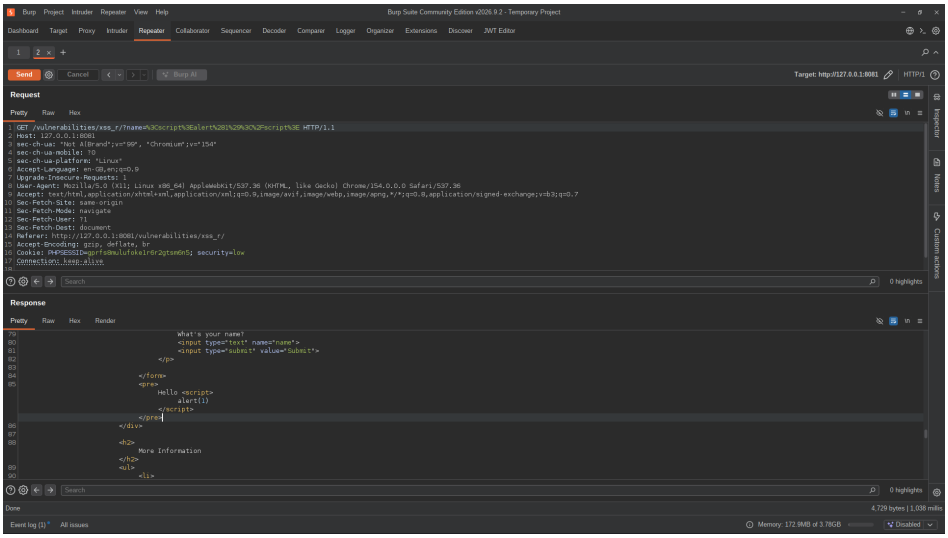


Figure 5. Repeater response showing the reflected input during the local test.

9. Finding 3 — Stored Cross-Site Scripting (XSS)

Target feature: DVWA Stored XSS guestbook, `/vulnerabilities/xss_s/`
Parameters observed: `txtName`, `mtxMessage`, and `btnSign`
Assessment result: Confirmed in local DVWA.

Description

Stored XSS occurs when untrusted content is saved by an application and later rendered to users without safe output encoding. Unlike reflected XSS, the payload does not need to be repeated in each viewer's request after it has been stored. In the local DVWA guestbook test, a guestbook entry was submitted and the browser displayed an alert reading "STORED XSS SUCCESSFUL." The entry was associated with the guestbook page, demonstrating script execution from stored content in the training environment.

Reproduction summary

1. Open the local Stored XSS guestbook and intercept the POST request.
2. Identify the name and message fields in the form body.
3. Submit the harmless alert proof-of-concept in the local lab only.
4. Confirm the browser alert appears when the stored entry is rendered.
5. Reload the page and check that the guestbook entry remains and the proof-of-concept still triggers, if persistence is part of the saved evidence.

Impact

On a real multi-user application, stored XSS can affect every user who views the stored content, making it potentially more persistent and broadly impactful than a reflected test. Risk depends on who can submit content, who views it, what privileges viewers hold, and whether browser/session protections limit the consequences. The evidence here is limited to DVWA and does not imply any external user was affected.

Remediation

Encode untrusted content at output according to its HTML context, sanitize rich text with a well-maintained allowlist sanitizer when rich formatting is genuinely required, and avoid unsafe DOM insertion. Apply server-side validation, review all rendering paths for stored content, and add regression tests that verify saved script-like input is displayed inertly after both submission and reload.

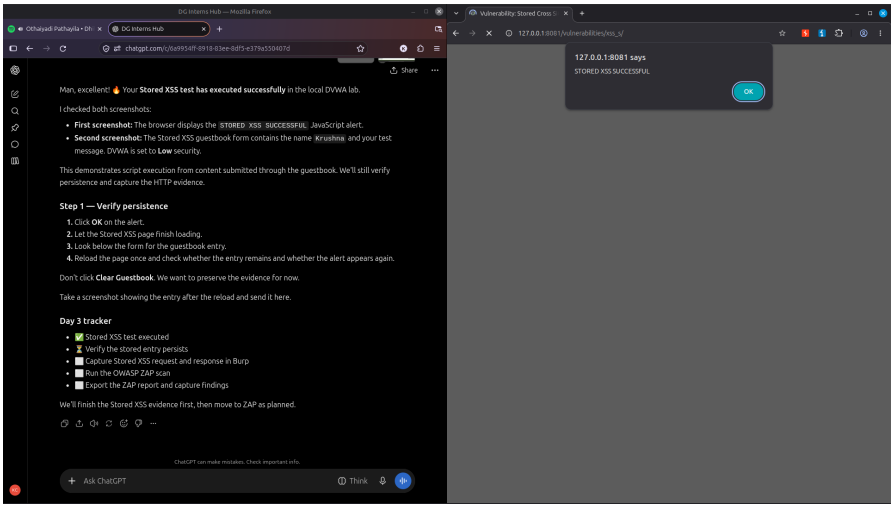


Figure 6. Local DVWA stored-XSS alert evidence.

10. Stored XSS — Persistence and Evidence Interpretation

The stored-XSS evidence is distinct from the reflected-XSS evidence because the test is submitted through a guestbook form and rendered as saved content. The alert proves that the browser executed the proof-of-concept during rendering. A separate persistence check—reloading the page and confirming the entry remains—helps establish that the content was retained rather than only echoed immediately in the submission response.

Request structure

The observed request used `POST /vulnerabilities/xss_s/` with the guestbook form fields `txtName`, `mtxMessage`, and `btnSign`. The POST method is consistent with submitting a new guestbook entry. The screenshot of the browser alert is proof of execution; the captured request documents the route and form fields used. Avoid publishing live cookies or tokens from the full request.

Why this is a separate finding

Although reflected and stored XSS share the same general class, they involve different data flows and remediation validation. Reflected XSS is tied to data returned from the current request, while stored XSS involves persistence and later rendering. The same core defensive principle—context-aware output encoding—applies, but stored content requires reviewing every page and component that displays the saved value.

Remediation verification checklist

- Submit script-like content into the name and message fields.
- Verify the entry is rendered as inert text or safely sanitized.
- Reload the page and revisit the entry to ensure it remains inert.
- Review other pages or administrative views that render the same stored data.
- Confirm normal guestbook content still works as intended.

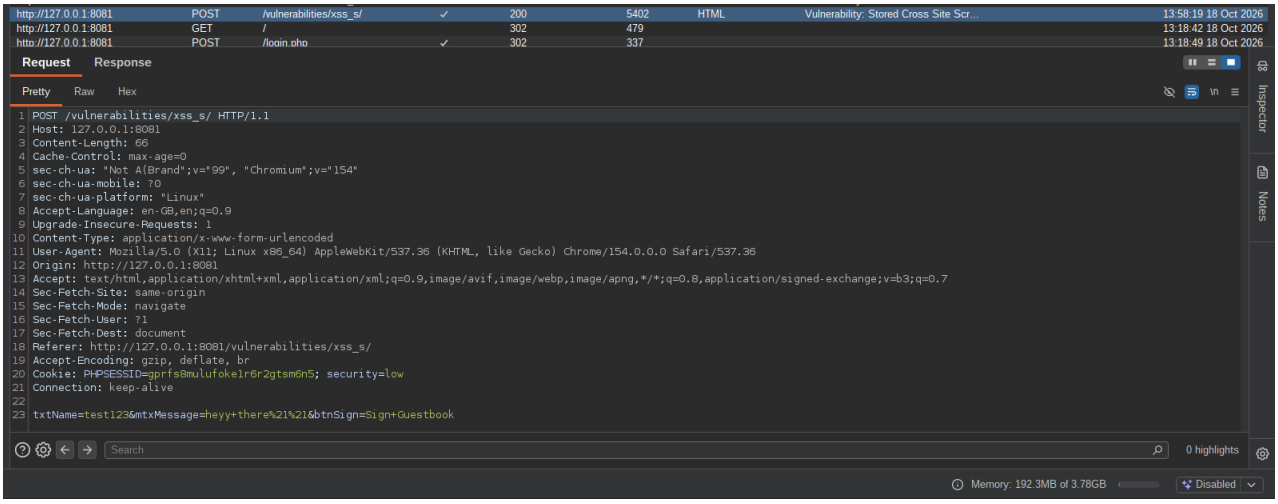


Figure 7. Captured Stored XSS guestbook POST request and form parameters.

11. Additional Tests — Parameter Tampering, Authentication and Intruder

Command module input influence

The command module request used the `ip` parameter. Changing the value from `127.0.0.1` to `127.0.0.2` changed the host shown in the ping output. This confirms that the supplied input influenced the operation, but it does not by itself prove arbitrary command injection. The report therefore does not count this observation as a confirmed fourth vulnerability. A production fix would avoid shell interpretation where possible, validate an IP address with a strict parser, invoke processes using argument arrays rather than shell strings, and enforce least privilege.

Authentication observation

The default credentials `admin/password` were accepted by the intentionally vulnerable local DVWA instance. This is recorded as a default-credential observation, not as an authentication bypass. A redirect or a successful login to a deliberately configured training account is not sufficient evidence of a bypass. Real applications should remove default accounts and require unique strong credentials.

Intruder exercise

Burp Intruder was used for a controlled two-payload run against the local test module. The recorded results showed HTTP 200 responses and differing response lengths. This verifies that the tool sent requests and returned results; response length differences alone do not establish a vulnerability. A valid finding requires a meaningful behavior change tied to a security impact.

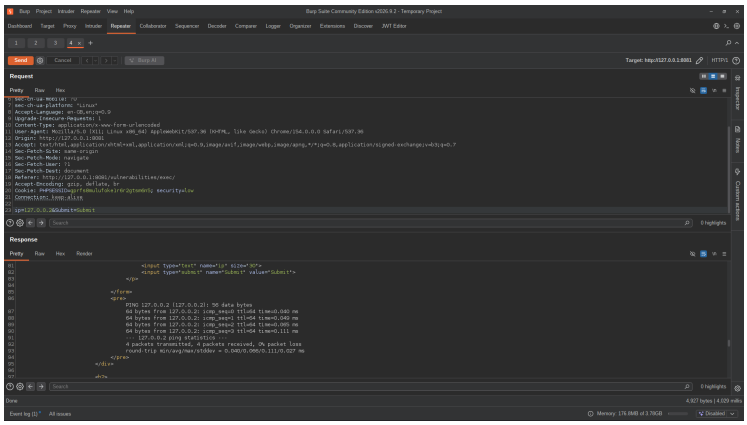


Figure 8. Command module input-tampering evidence; interpreted as input influence only.

12. Directory and Endpoint Discovery

A small Python path checker requested 14 predefined paths on the local DVWA host. The check was intentionally limited and should not be described as exhaustive. The result list contained several expected application pages and diagnostic or configuration-related paths. Status codes indicate how the server responded to each request; they do not automatically establish that a path is vulnerable.

Observed results

Status	Paths observed	Interpretation
200	robots.txt, login.php, index.php, setup.php, security.php, instructions.php, about.php, phpinfo.php, config/	The server returned a successful response. Review the actual content and access requirements before concluding there is a disclosure issue.
403	server-status	The request was forbidden. This is not proof of an exploitable weakness; access was denied for this request.
404	admin, api, uploads/, backup/	These paths were not found in this request context. They may be absent or routed differently.

Interpretation notes

`robots.txt` contains crawler guidance, including a disallow rule. Robots rules are not an access-control mechanism and should not be relied upon to protect sensitive paths. A 200 response for `phpinfo.php` or `config/` is a prompt to inspect whether sensitive information is exposed; the status code alone does not prove disclosure. The scan did not assess the full content of every returned path.

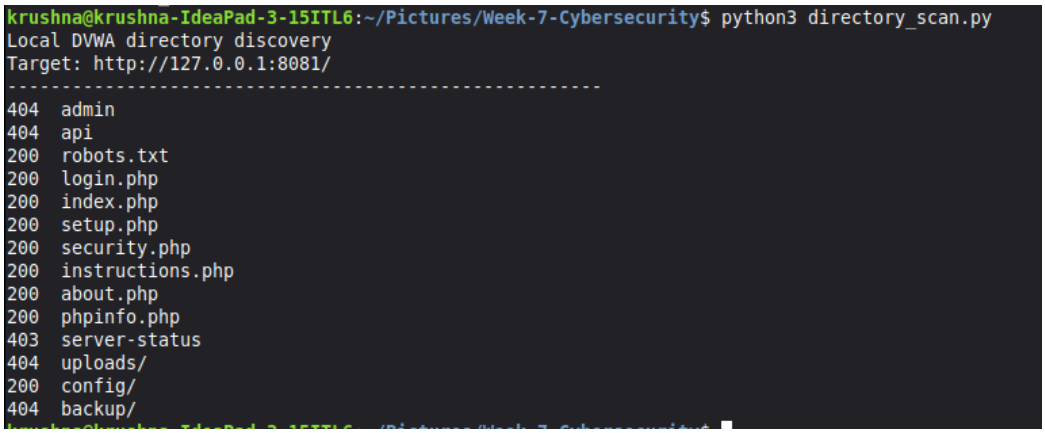


Figure 9. Directory/path discovery output from the local lab.

13. Risk Summary and Remediation Plan

The following prioritization is qualitative and intended for this training report. It is based on the demonstrated behavior in DVWA, not on a production risk assessment. Severity in a real system would depend on exposure, authentication requirements, affected users, data sensitivity, and exploitability.

Finding	Evidence	Priority in a comparable real app	Primary remediation
SQL Injection	SQL error and Boolean-style input changed returned records.	High, if exploitable against sensitive data or authentication logic.	Parameterized queries; least-privilege DB account; safe errors.
Reflected XSS	Browser alert executed from reflected request parameter.	Medium to High depending on victim interaction and privileges.	Context-aware output encoding; safe DOM APIs; CSP as defense in depth.
Stored XSS	Stored guestbook content triggered browser alert when rendered.	Medium to High; potentially broader reach where many users view content.	Encode output; sanitize rich text; test all views of stored content.

Recommended order of work

1. Fix SQL injection first with prepared statements and test all database-backed inputs.
2. Fix reflected and stored XSS with context-aware output encoding; review every output path, including stored content.
3. Remove or change default credentials and ensure diagnostic pages are not exposed in real deployments.
4. Review server-side command execution patterns, validate inputs and avoid shell invocation.
5. Re-run the same tests after changes and retain before/after evidence.

General defensive controls

Keep application dependencies updated, apply least privilege to database and OS accounts, centralize security logging, and use automated regression tests for injection and output encoding. Security headers are useful defense in depth but do not replace correcting unsafe query construction or output handling.

14. Conclusion, Limitations and Evidence Index

This exercise practiced the main stages of a small web-application VAPT workflow: intercepting traffic, mapping routes and parameters, testing controlled inputs, and recording observations in a structured format. Three distinct behaviors were demonstrated in local DVWA: SQL injection, reflected XSS, and stored XSS. Additional command-module, authentication, Intruder, and directory-discovery tests were documented with conservative conclusions where evidence did not establish a vulnerability.

Limitations

The target was a deliberately vulnerable local application at low security. The work was not a comprehensive assessment, did not test production systems, and does not establish exploitability outside this lab. Directory discovery covered only 14 predefined paths. The command-module test showed input influence but not arbitrary command execution. Default credentials were accepted by DVWA but were not evidence of authentication bypass. HTTP status codes and response-length changes were not treated as standalone vulnerability proof.

Evidence index

• Burp listener / proxy configuration: `01b_Burp-Proxy-Listener.png`
• SQLi GET request and result: `06_Task23-SQLi-GET-Request.png`, `10_SQLi-Boolean-Bypass-Result.png`, `SQLi-Burp-Response.png`
• Reflected XSS: `07_Task23-Reflected-XSS-GET-Request.png`, `06_Reflected-XSS-Repeater-Response.png`, `07_Reflected-XSS-Alert.png`
• Stored XSS: `08_Task23-Stored-XSS-POST-Request.png`, `08_Stored-XSS-Page.png`
• Additional tests: `13_Task24-Parameter-Tampering-Command-Injection.png`, `14_Task24-Intruder-Two-Payload-Results.png`, `15_Task24-Default-Credentials-Success.png`
• Discovery: `17_Task25-Robots-txt-Disallow-Root.png`, `18_Task25-Robots-txt-Burp-Evidence.png`, `19_Task25-Directory-Discovery-Scan-Results.png`

Final note

Before uploading or posting evidence publicly, redact session cookies, tokens, and any credentials visible in screenshots. Re-test after remediation rather than assuming a fix is effective.

Prepared as an educational deliverable for DG Interns Hub — Week 7.